Q4 – Thread Creation and the Thread Life Cycle

Shared code

```
class StationThread extends Thread {
    private final int scripts; static int total = 0;
    StationThread(String name, int scripts) { super(name);
this.scripts = scripts; }
    public void run() {
        System.out.println(getName() + " started");
        try { Thread.sleep(100 + (int)(Math.random() * 400)); } catch
(InterruptedException e) {}
        synchronized (StationThread.class) { total += scripts; }
        System.out.println(getName() + " scanned " + scripts);
    }
}

class StationTask implements Runnable {
    private final int scripts;
    StationTask(int scripts) { this.scripts = scripts; }
    public void run() {
        System.out.println(Thread.currentThread().getName() + "
scanned " + scripts);
    }
}

public class Scanners {
    public static void main(String[] args) throws InterruptedException
{
        StationThread[] ts = new StationThread[4];
        for (int i = 0; i < 4; i++) { ts[i] = new
StationThread("Station-" + (i + 1), 100 * (i + 1)); ts[i].start(); }
        System.out.println("Station-1 alive? " + ts[0].isAlive());
        for (StationThread t : ts) t.join();
        System.out.println("Total (Thread version) = " +
StationThread.total);

        Thread[] rs = new Thread[4];
        for (int i = 0; i < 4; i++) { rs[i] = new Thread(new
StationTask(100 * (i + 1)), "Runner-" + (i + 1)); rs[i].start(); }
        for (Thread t : rs) t.join();
        System.out.println("All stations finished");
    }
}
```

Student A

- **Thread vs Runnable:** extending `Thread` ties the task to the thread class and uses up your single inheritance. Implementing `Runnable` separates the task from the execution

mechanism, lets the class extend something else, and lets you reuse the task with thread pools. Prefer `Runnable` (or `Callable`) almost always. Extend `Thread` only when you need to change the thread's own behaviour.

- **Life-cycle:** `NEW → (start) → RUNNABLE ⇄ (scheduler) RUNNING → [TIMED_WAITING (sleep/join with timeout) | WAITING (join/wait) | BLOCKED (waiting for monitor)] → RUNNABLE → TERMINATED`. Each station: NEW → RUNNABLE → TIMED_WAITING (sleep) → RUNNABLE → TERMINATED. Print `getState()` at each stage as evidence.
- **run() vs start():** `start()` creates a new call stack and calls `run()` in a new thread. Calling `run()` is an ordinary method call on the current thread. Evidence: print `Thread.currentThread().getName()` inside `run()`, which shows `main` when `run()` is called directly and `Thread-0/Station-1` with `start()`.
- **join():** `t.join()` makes the calling thread (main) wait until `t` terminates. Joining all four before printing the total guarantees every station has added its count.

Student B – Experiments (record in a table)

Run	Order of finishing
1...10	`<fill>`

- **Priorities:** `setPriority(Thread.MAX_PRIORITY / MIN_PRIORITY)`. Priorities are only hints; on most OSes the order does not reliably change, so record exactly what you see.
- **Daemon:** `t.setDaemon(true)` before `start()`. When `main` ends and only daemon threads remain, the JVM exits, and the daemon may be killed before finishing (its output might not appear).
- **Explanation:** the OS/JVM scheduler decides which runnable thread gets the CPU (time-slicing, number of cores, load), so order is non-deterministic.

Joint conclusion

Thread scheduling depends on the OS, CPU load and timing, none of which the program controls. A correct program must use synchronization/join (not sleep guesses or assumed ordering) to be correct under every interleaving.

Alternative: ExecutorService

```
ExecutorService ex = Executors.newFixedThreadPool(4);
List<Future<Integer>> fs = new ArrayList<>();
for (int i = 1; i <= 4; i++) { final int n = 100 * i;
fs.add(ex.submit(() -> n)); }
int total = 0; for (Future<Integer> f : fs) total += f.get();
ex.shutdown();
```

- **Time:** n scripts, p threads, each script takes time c: about $\lceil n/p \rceil \times c$, i.e. $O(n/p)$ (plus thread start overhead). With one thread per batch/station you get the same parallelism only if stations ≤ cores.

- **Space:** each platform thread has a stack (typically ~1 MB reserved). Manual creation: one thread per task, so O(n × stack). Pool: O(p × stack), tasks only cost small queue objects. Pool also reuses threads, avoiding creation cost.